Desarrollo de Pulso con SDD

Informe grupal sobre el proceso y la validación del proyecto
29 de octubre de 2026

El proyecto y las decisiones iniciales

El trabajo consistió en desarrollar una página para consultar criptomonedas utilizando CoinGecko y el flujo de GitHub Spec Kit. La llamamos Pulso. Desde ella se puede buscar un activo por nombre o símbolo, ver su precio en dólares, consultar la variación y el volumen de las últimas 24 horas, y explorar su comportamiento en gráficos de distintos períodos. También incorporamos favoritos para seguir monedas sin tener que buscarlas cada vez.

Antes de programar tuvimos que precisar el alcance. El enunciado mencionaba criptomonedas y acciones, pero la API elegida estaba orientada a los datos cripto que necesitábamos. Por eso centramos la entrega en criptomonedas como BTC y ETH. Si alguien busca AAPL, la página explica que corresponde a una acción bursátil fuera de ese alcance. No tendría sentido mostrar un token parecido como si fuera la cotización de Apple.

Para organizar el trabajo usamos SDD, es decir, desarrollo guiado por especificaciones. La idea fue dejar por escrito qué debía hacer la página y usar esas reglas para revisar el resultado. En lugar de pedir solamente “hacer un rastreador”, definimos comportamientos concretos: búsqueda sin distinguir mayúsculas, precios en USD, cuatro temporalidades, favoritos persistentes y mensajes cuando la API no responde.

El artículo de Piskala (2026) nos dio una referencia para entender ese proceso. Propone especificar, planificar, implementar y validar, con revisión humana entre etapas. Al hablar de Kiro, describe la preparación de requisitos, diseño y tareas antes de generar código. En este proyecto usamos GitHub Spec Kit, no Kiro, pero tomamos esa secuencia como guía para analizar el trabajo realizado.

Los requisitos quedaron en spec.md y se agruparon en cuatro historias de usuario: consultar el mercado, analizar históricos, guardar favoritos y documentar la entrega. Se definieron 13 requisitos funcionales y cinco condiciones de éxito. Esto permitió comprobar asuntos que podrían pasar desapercibidos al mirar solo la interfaz, como diferenciar un dato ausente de un valor igual a cero o identificar una captura guardada cuando no hay conexión.

Cómo desarrollamos la página

Después de definir los requisitos elaboramos el plan técnico. Elegimos HTML, CSS y JavaScript sin un framework porque el alcance no requería cuentas, transacciones ni una base de datos. La información viene de CoinGecko y los favoritos se guardan en el navegador. Esta decisión hace que el proyecto se pueda ejecutar con Node.js sin instalar dependencias de producción.

Separamos la lógica en dos archivos principales. core.js contiene las consultas a la API, el formato de los valores, la limpieza de las series y los cálculos. app.js controla la búsqueda, la selección del activo, la tabla, los favoritos y los gráficos. El plan, el modelo de datos y el contrato de los endpoints se conservaron junto con las tareas para que las decisiones pudieran revisarse después.

Instalamos Specify CLI 1.2.5 e inicializamos la integración de Codex con los scripts de PowerShell. Se guardaron las salidas de instalación y verificación. Hubo una diferencia respecto al orden ideal: los requisitos y el plan ya estaban escritos antes del código, pero la instalación terminó cuando la implementación había comenzado. La constitución formal del proyecto se completó entonces. En una nueva ejecución convendría cerrar esa preparación antes de programar.

La construcción empezó por las consultas y los cálculos, y continuó con la tabla de mercado y el detalle de cada moneda. Después añadimos los gráficos de 24 horas, 7, 30 y 90 días. Cada rango solicita un histórico distinto; no cambia únicamente el aspecto del mismo dibujo. Los gráficos permiten alternar precio y volumen, consultar una muestra con el cursor y recorrer valores con el teclado.

Una decisión importante fue separar la variación de 24 horas del rendimiento del período seleccionado. Si el usuario mira 30 días, el cambio del gráfico se calcula entre la primera y la última muestra de esos 30 días. El volumen también tiene un significado específico: CoinGecko devuelve volumen móvil de 24 horas, por lo que no sumamos los puntos como si cada uno correspondiera a operaciones independientes.

Para reducir consultas repetidas añadimos caché y una espera después de recibir un error 429. También guardamos una captura real de 50 activos y los cuatro históricos de Bitcoin. Si falla el proveedor, la página puede mostrar ese respaldo con su fecha. No se generan precios de ejemplo para llenar el gráfico. Finalmente se publicó la página en Sites con acceso privado inicial y se conservó la confirmación del despliegue.

Revisión del trabajo y del código generado

La revisión se organizó con las fases del artículo. En requisitos, el punto principal fue comprobar si habíamos entendido el encargo. La decisión que necesita confirmar el grupo es el alcance de criptomonedas: si se exigen también acciones de bolsa, habría que modificar la especificación y acordar otro proveedor. También deben quedar claros los rangos, la moneda y el comportamiento ante errores antes de dar por aceptado el producto.

En diseño, revisamos si las decisiones técnicas eran suficientes para resolver esas necesidades. Guardar favoritos en el navegador es adecuado para esta entrega, aunque implica que no se sincronizan entre equipos. Consultar CoinGecko desde el cliente evitó añadir un servidor de aplicación, pero nos dejó sujetos a las restricciones del acceso público. Si más adelante se utiliza una clave privada, habría que manejarla desde un servidor, no incluirla en la página.

En implementación, la discusión se centró en lo que produjo la IA y en cómo comprobarlo. Spec Kit aportó las guías, las plantillas y la estructura del proceso; el código de la página se construyó con asistencia de IA y recibió ajustes directos. Por eso no describimos el proyecto como un sistema que regenera automáticamente todo el código desde una especificación. Los archivos y las pruebas siguen necesitando mantenimiento.

La separación de los cálculos permitió revisarlos con ejemplos sencillos. Con una serie de precios de 100, 80 y 120, la función devuelve un mínimo de 80, un máximo de 120 y un rendimiento cercano al 20 %. Si el primer precio es cero, evita dividir por cero y deja el rendimiento como dato ausente. Esto es más útil para evaluar el código que asumir que una fórmula está bien solo porque el gráfico parece correcto.

Otro punto fue el manejo de peticiones tardías. Al cambiar rápidamente de moneda o temporalidad, una respuesta anterior podría llegar después de la nueva. El código identifica cada consulta con una secuencia y descarta la que ya no corresponde a la selección vigente. Se revisó ese mecanismo y se probaron cambios rápidos, aunque todavía falta una prueba automática de interfaz que fuerce distintos órdenes de respuesta.

La revisión técnica que documentamos fue asistida por IA y se apoya en pruebas y evidencias del proyecto. La aprobación humana final corresponde a los cuatro integrantes. Para cerrarla, el grupo debe recorrer la página, revisar estas decisiones y registrar las observaciones que tenga. Esa aceptación posterior no debe presentarse como si hubiera ocurrido antes de cada fase. El artículo insiste en conservar el juicio humano incluso cuando las pruebas pasan.

Validación y balance del resultado

La validación combinó pruebas automatizadas, inspección del código y comprobaciones de navegador. Para este informe se volvió a ejecutar el conjunto de 20 pruebas y la revisión de sintaxis de los módulos: todas las pruebas aprobaron. Se comprobaron rangos, cálculos, búsquedas, favoritos, caché, consultas simultáneas, timeout y respuestas de error. Los errores HTTP se simularon para poder repetir los casos sin depender de que CoinGecko fallara en ese momento.

En la revisión de navegador realizada durante la construcción se comprobó la búsqueda de BTC y ETH, la selección de Ethereum, el cambio de temporalidades y la vista de volumen. También se verificó que un favorito permaneciera después de recargar, que el gráfico respondiera al teclado y que AAPL recibiera una explicación. Durante el uso intensivo apareció un límite real del proveedor y se observó el paso a la captura fechada.

Estas comprobaciones llevaron a corregir problemas concretos. En móvil, la página tenía un desbordamiento horizontal: el contenido ocupaba 492 píxeles en un área de 375. Se ajustó la tabla y el ancho de los nombres; después ambos valores quedaron en 375. También se corrigió la recuperación de favoritos que no estaban dentro del primer listado del mercado.

La revisión adicional encontró un caso que no cubrían las 20 pruebas. El formato de precios usa como máximo seis decimales para valores menores a un dólar, de modo que 0,00000001 USD termina mostrándose como $0.0000. Es una limitación pendiente: habría que adaptar los decimales a la magnitud del precio y añadir una prueba para evitar que un valor positivo parezca cero. Que las pruebas actuales aprueben no elimina este problema.

El resultado cumple el núcleo del rastreador propuesto y permite demostrar el recorrido desde los requisitos hasta una página funcional. Lo que falta fortalecer es la validación automática de la interfaz, la precisión de precios muy pequeños y la aceptación final del equipo. También aprendimos que documentar las fases no basta: las revisiones deben ocurrir en el momento previsto y sus decisiones deben quedar registradas. En este trabajo, SDD nos sirvió sobre todo para explicar qué se construyó, por qué se tomaron ciertas decisiones y qué evidencia respalda el resultado.

Referencias

Piskala, D. B. (2026). Spec-Driven Development: From Code to Contract in the Age of AI Coding Assistants. arXiv:2602.00180v1. Se consultaron las pp. 3-5 sobre el flujo, la revisión humana y Kiro. https://doi.org/10.48550/arXiv.2602.00180

GitHub. Spec Kit. https://github.com/github/spec-kit. CoinGecko. API. https://www.coingecko.com/en/api.

Proyecto Pulso (2026). Especificación, plan, código y evidencias: specs/001-crypto-tracker/, tests/core.test.mjs y docs/evidence/. Página: https://pulso-mercado-sdd.deepgiant66.chatgpt.site.